

On Redundancy in Alloy Models

Soaibuzzaman, Jan Oliver Ringert

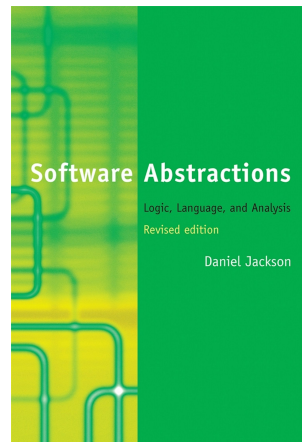
Bauhaus-University Weimar, Germany



Bauhaus-
Universität
Weimar

Alloy Language and Alloy Analyzer

- Alloy: a modeling language based on relational first-order logic and temporal logic
 - Everything is a relation
 - Structure: signatures and fields
 - Constraints: facts, predicates, commands
- Alloy Analyzer: finds instances and counterexamples in a bounded scope – quick, interactive feedback
- Applications: software design models, API design, protocol and security analyses, software synthesis . . .



Example: Teaching Modules

- Profs teach modules (l. 8)
Inverse relations (l. 10)
Every module is taught (l. 11)

```
1 abstract sig Person { teaches : set Module }
2 sig Professor extends Person {}
3 sig Module {
4   teacher : Person // default multiplicity one
5 }
6
7 fact teachersAndTeaching {
8   Module.teacher = Professor} // profs teach modules
9 fact teachersOfModules {
10   teacher = ~teaches // teachers teach their modules
11   Professor.teaches = Module} // no dangling modules
12 run {some Person} for 2 // browse example scenarios
```

Example: Teaching Modules

- Profs teach modules (l. 8)
Inverse relations (l. 10)
Every module is taught (l. 11)
- Students don't teach (l. 17)
Show teaching lecturers (l. 18)

```
1 abstract sig Person { teaches : set Module }
2 sig Professor extends Person {}
3 sig Module {
4   teacher : Person // default multiplicity one
5 }
6
7 fact teachersAndTeaching {
8   Module.teacher = Professor} // profs teach modules
9 fact teachersOfModules {
10   teacher = ~teaches // teachers teach their modules
11   Professor.teaches = Module} // no dangling modules
12 run {some Person} for 2 // browse example scenarios
13
14 // extended later with additional signatures
15 sig Student, Lecturer extends Person {}
16 fact studentsDontTeach {
17   no s : Student | some m : Module | m in s.teaches}
18 run {some Lecturer.teaches} // show teaching lecturers
```

Example: Teaching Modules

- Profs teach modules (l. 8)
Inverse relations (l. 10)
Every module is taught (l. 11)
- Students don't teach (l. 17)
Show teaching lecturers (l. 18)
- l. 18 **no instance!**
- Deleting l. 11 alone does not help
– l. 8 must change too
- l. 8, l. 10 $\models$ l. 11

```
1 abstract sig Person { teaches : set Module }
2 sig Professor extends Person {}
3 sig Module {
4   teacher : Person // default multiplicity one
5 }
6
7 fact teachersAndTeaching {
8   Module.teacher = Professor} // profs teach modules
9 fact teachersOfModules {
10   teacher = ~teaches // teachers teach their modules
11   Professor.teaches = Module} // no dangling modules
12 run {some Person} for 2 // browse example scenarios
13
14 // extended later with additional signatures
15 sig Student, Lecturer extends Person {}
16 fact studentsDontTeach {
17   no s : Student | some m : Module | m in s.teaches}
18 run {some Lecturer.teaches} // show teaching lecturers
```

 redundant  explanation

Redundancy in Alloy Models

- $\text{cons} : \text{Model} \rightarrow 2^{\text{Expr}}$ extracts all constraints from an Alloy model m
- $\text{rem} : \text{Model} \times 2^{\text{Expr}} \rightarrow \text{Model}$ produces a new model by removing constraints

Constraint $c \in \text{Expr}$ is redundant if removing it does not change the instances of model m .

Local redundancy (one command)

For all instances under command cmd ,
 $\llbracket m \rrbracket_{\text{cmd}} = \llbracket \text{rem}(m, c) \rrbracket_{\text{cmd}}$

```
7 fact teachersAndTeaching {  
8   Module.teacher = Professor} // profs teach  
9 fact teachersOfModules {  
10   teacher = ~teaches           // inverse  
11   Professor.teaches = Module} // all taught  
12 run {some Person} for 2  
13 run {no Person} for 2         // new command
```

Global redundancy (all commands)

$\forall \text{cmd} \in m.\text{cmds} : \llbracket m \rrbracket_{\text{cmd}} = \llbracket \text{rem}(m, c) \rrbracket_{\text{cmd}}$

Redundant for...	I. 8	I. 10	I. 11
run {some Person}			✓
run {no Person}	✓	✓	✓
all commands			✓

No persons $\Rightarrow$ no teachers, no modules $\Rightarrow$ every fact holds trivially.
I. 8 and I. 10 are only *locally* redundant; I. 11 is *globally* redundant.

Maximal Redundant Subsets

- For two redundant constraints c_1 and c_2 ,
 $\llbracket \text{rem}(m, c_1) \rrbracket = \llbracket m \rrbracket = \llbracket \text{rem}(m, c_2) \rrbracket$, but
potentially $\llbracket \text{rem}(m, \{c_1, c_2\}) \rrbracket \neq \llbracket m \rrbracket$

Maximal Redundant Subsets

- For two redundant constraints c_1 and c_2 ,
 $\llbracket \text{rem}(m, c_1) \rrbracket = \llbracket m \rrbracket = \llbracket \text{rem}(m, c_2) \rrbracket$, but
potentially $\llbracket \text{rem}(m, \{c_1, c_2\}) \rrbracket \neq \llbracket m \rrbracket$
 - Remove l. 5 *or* l. 6 alone
 - Remove l. 5 *and* l. 6 at once

```
1 abstract sig Person {teaches: set Module}
2 sig Professor, Student extends Person {}
3 sig Module { teacher : Person }
4 fact studentsDontTeach {
5   no s : Student | some s.teaches }
6 fact { no Student.teaches } // same
7 run {some Person} for 2
```

Maximal Redundant Subsets

- For two redundant constraints c_1 and c_2 , $\llbracket \text{rem}(m, c_1) \rrbracket = \llbracket m \rrbracket = \llbracket \text{rem}(m, c_2) \rrbracket$, but potentially $\llbracket \text{rem}(m, \{c_1, c_2\}) \rrbracket \neq \llbracket m \rrbracket$
 - Remove l. 5 or l. 6 alone
 - Remove l. 5 and l. 6 at once

```
1 abstract sig Person {teaches: set Module}
2 sig Professor, Student extends Person {}
3 sig Module { teacher : Person }
4 fact studentsDontTeach {
5   no s : Student | some s.teaches }
6 fact { no Student.teaches } // same
7 run {some Person} for 2
```

Maximal Redundant Subset $C \subseteq \text{cons}(m)$

For a set of constraints $C \subseteq \text{cons}(m)$

- Removing C preserves semantics: $\forall \text{cmd} \in m.\text{cmds} : \llbracket m \rrbracket_{\text{cmd}} = \llbracket \text{rem}(m, C) \rrbracket_{\text{cmd}}$
- C is maximal for the above property

Checking Redundancy in Alloy Models

- Construct Alloy model $m' = \text{rem}(m, \text{cons}(m))$ (m with all constraints removed)
- A new run command cmd' from command cmd where

$$cmd'.pred = cmd.pred \wedge \bigwedge (\text{cons}(m) \setminus C) \wedge \neg \bigwedge C$$

UNSAT $\rightarrow C$ are redundant for m and cmd

Checking Redundancy in Alloy Models

- Construct Alloy model $m' = \text{rem}(m, \text{cons}(m))$ (m with all constraints removed)
- A new run command cmd' from command cmd where

$$\text{cmd}'.\text{pred} = \text{cmd}.\text{pred} \wedge \bigwedge (\text{cons}(m) \setminus C) \wedge \neg \bigwedge C$$

UNSAT $\rightarrow C$ are redundant for m and cmd

- **Maximal Redundant Subset:** delta debugging (*DDMin*) shrinks the set of *needed* constraints $\text{cons}(m) \setminus C$
 - UNSAT $\rightarrow C$ is still redundant, shrink further
 - SAT $\rightarrow C$ is not redundant, try another split

Explanation of a redundant constraint c

A minimal set of constraints $E \subseteq \text{cons}(m) \setminus \{c\}$ that makes c redundant (like l. 8 and l. 10 in our example)

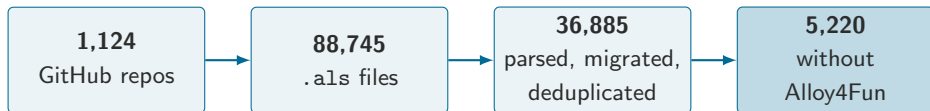
- *DDMin* shrinks E , starting from all other constraints $\text{cons}(m) \setminus \{c\}$
- Check for a candidate E :

$$cmd'.pred \leftarrow \bigwedge E \wedge cmd.pred \wedge \neg c$$

- UNSAT $\rightarrow E$ still makes c redundant, shrink further
- SAT $\rightarrow E$ is too small, try another split

- **RQ1** What is the prevalence of redundancy in Alloy models?
- **RQ2** What are the sizes of maximally redundant subsets?
- **RQ3** What are the sizes of explanations?
- **RQ4** What is the cost of computing maximally redundant subsets and explanations?
- **RQ5** What is the performance impact of globally redundant constraints on analyses?

Evaluation: Dataset



- Alloy4Fun: instructor-written core, one student predicate per command
- Typical (median) model: 68 lines, 5 signatures, 5 flat facts ($|cons(m)|$), 1 command

RQ1 + RQ2: Prevalence of Redundancy & Sizes of Max. Redundant Subsets

40.9%

local redundancy

2,108 / 5,220

26.6%

global redundancy

1,388 / 5,220

RQ1 + RQ2: Prevalence of Redundancy & Sizes of Max. Redundant Subsets

40.9%

local redundancy

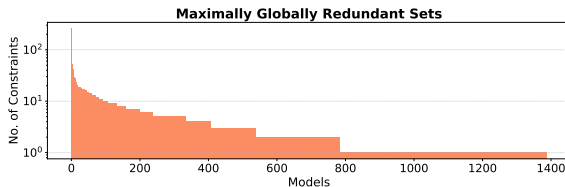
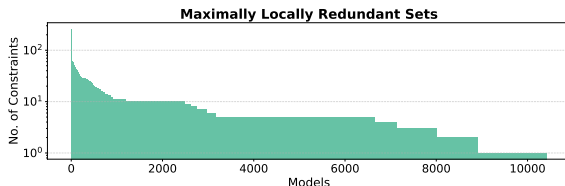
2,108 / 5,220

26.6%

global redundancy

1,388 / 5,220

- Usually few: < 10 redundant constraints
- Some spanning over dozens to hundreds – mostly in tool-generated models
- Not limited to trivial, single-line duplicates

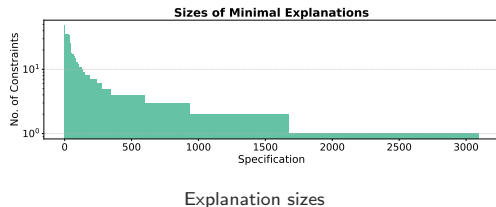


Sizes of maximal locally (top) and globally (bottom) redundant sets
(log scale, includes Alloy4Fun benchmark)

RQ3: Sizes of Explanations of Redundancy

- (*DDMin*) Searches **constraints only**
 - Short, mostly < 10 constraints
- 1,064 of 4,157 general explanations *empty*
 - *DDMin*-based explanation **empty** – no other *fact* explains l.3
 - Structural redundancy

```
1 sig Person {}  
2 sig Module { teacher : Person }  
3 fact { all m : Module | some m.teacher }
```



RQ4: Cost of Computing Maximally Redundant Subsets and Explanations

- Maximal sets: about $2\times$ the time of checking constraints one by one
 - But give more information
- Fast in practice
 - 75% of local analyses < 1.3 s
- Timeouts (10min) in only $\sim 3\%$ of models
- Explanations: stable ~ 2 s (median)

	Median	75%
Max. redundant set	0.68 (s)	1.22 (s)
Explanation	1.99 (s)	2.40 (s)

RQ5: Performance impact of globally redundant constraints on analyses

- Each model's first command with and without its redundant constraints

	Solving time	Primary vars	All SAT vars
25%	1.43	1.00	1.00
Median	1.79	1.00	1.06
75%	2.30	1.00	1.61

Ratio: with redundant constraints / without (first command)

1.79×

slower analysis
(median)

- Ratio greater than 1.0 indicates that redundancy increases solving time
- Keeping redundant constraints makes the analysis slower
- Almost always the same primary variables – but more auxiliary variables in the CNF

- **Task 1:** extend the example with
Lecturer
 - **5 of 6** changed only one of the two linked facts → no lecturer instances
- **Tasks 2–3:** find redundancy by hand
 - Fact-to-fact: hard for novices
 - Structural: **nobody** found it unprompted
- Tool: useful and easy; explanation is a *starting point*

"... it's just more places you have to update."

– E1

"[There is] a degree of safety in writing a specification to introduce something that's redundant and check that it's redundant."

– E1

Takeaways

- Redundancy is **common**: 41% of models locally, 27% globally
- Redundancy is **hard to find by hand** – tool support helps
- We **detect** redundant constraints, compute **maximal redundant subsets**, and **explain** redundancy – inside the Alloy Analyzer
- Explanations: *DDMin* is short; structural redundancy needs better explanations
- Not always bad: *intended* redundancy can confirm your understanding
- Redundancy has a **cost**: $1.79\times$ slower analysis (median), harder maintenance

Tool, Dataset, and Docker replication package:

<https://se-buw.de/publication/alloy-redundancy-models-2026/>



